TRABAJO PRÁCTICO 1B

Nombre: Mia Bertola - Julieta Barahona Roth

*Como tenemos dos sistemas operativos diferentes, hicimos juntas la teoría pero cada una adjunta sus códigos.

CONSIGNAS:

1. Utilizando el código "sniffer" que es un programa que muestra el contenido del tráfico que llega, modificarlo para explicar cada paso que va ejecutando en pantalla.
2. Modificar el sniffer para que vaya mostrando todo el tráfico que llega.
3. Enviar tráfico al "sniffer" desde el cliente escrito en la parte A del TP1
4. Enviar tráfico ICMP al "sniffer" y mostrar los resultados con un LOG con comentarios.
5. Explicar las estructuras del código de ICMP y del Sniffer, especialmente los campos de protocolo.
6. Estudiar el "sniffer" para que muestre IP, TCP y UDP.
7. Mostrar resultados.

RESPUESTAS:

1. Paso a paso explicando el código:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
#include <arpa/inet.h>


int main() {
   int sockfd;
   char buffer[65536];
   struct sockaddr saddr;
   socklen_t saddr_len = sizeof(saddr);
   printf("Paso 1: Creo el raw socket para capturar trafico ICMP\n");
```

```
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);
    if (sockfd < 0) {
        perror("\nError al crear el socket");
        return 1;
    }


    printf("Paso 2: Socket creado correctamente.\n");

    printf("\n\nSniffer ICMP iniciado... (Ctrl+C para detener)\n\n");

    printf("Paso 3: El sniffer queda esperando la llegada de paquetes...\n");
    while (1) {
        printf("\nPaso 4: Esperando un paquete con recvfrom() \n");
        ssize_t packet_size = recvfrom(sockfd, buffer, sizeof(buffer), 0, &saddr,
&saddr_len);
        if (packet_size < 0) {
            perror("recvfrom error");
            continue;
        }

        printf("Paso 5: Paquete recibido correctamente.\n");
        printf("Paso 6: Tamaño del paquete recibido: %zd bytes.\n", packet_size);
        printf("Paso 7: Interpretando el comienzo del buffer como cabecera IP\n");

        struct ip *ip = (struct ip *)buffer;
        printf("Paso 8: Leyendo el campo protocolo de la cabecera IP\n");
        printf("\nProtocolo detectado en IP: %d\n\n", ip->ip_p);

        if (ip->ip_p == IPPROTO_ICMP) {
            printf("Paso 9: el protocolo detectado es ICMP.\n");
            printf("Paso 10: Calculando donde comienza la cabecera ICMP\n");

            struct icmp *icmp = (struct icmp *)(buffer + ip->ip_hl * 4);
            printf("Paso 11: Leyendo los campos de la cabecera ICMP\n");
            printf("\n\nICMP capturado: Tipo=%d Código=%d ID=%d Secuencia=%d\n\n",
                icmp->icmp_type,
                icmp->icmp_code,
                ntohs(icmp->icmp_id),
                ntohs(icmp->icmp_seq)
            );
            printf("Paso 12: guardando el paquete en el archivo icmp_packet.bin\n");
            FILE *f = fopen("icmp_packet.bin", "wb");
            if (f) {
                fwrite(buffer, 1, packet_size, f);
```

```
                fclose(f);
                printf("Paso 13: paquete guardado en icmp_packet.bin\n");
                printf("\n\nPaquete guardado en icmp_packet.bin\n\n");
                break; // solo capturamos uno para el ejemplo
            }

            printf("Paso 14: Finalizando.\n");

        }
    }
    printf("Paso 15: Cerrando el socket\n");
    close(sockfd);
    printf("Paso 16: Programa finalizado correctamente.\n");
    return 0;
}
```

```
tp1B — -zsh — 97×36
julietabarahonaroth@Julietas-MacBook-Air tp1B % gcc raw_socket_sniffer_explicado.c -o sniffer_exp
licado

julietabarahonaroth@Julietas-MacBook-Air tp1B % sudo ./sniffer_explicado
Paso 1: Creo el raw socket para capturar trafico ICMP
Paso 2: Socket creado correctamente.


Sniffer ICMP iniciado... (Ctrl+C para detener)

Paso 3: El sniffer queda esperando la llegada de paquetes...

Paso 4: Esperando un paquete con recvfrom()
Paso 5: Paquete recibido correctamente.
Paso 6: Tamaño del paquete recibido: 576 bytes.
Paso 7: Interpretando el comienzo del buffer como cabecera IP
Paso 8: Leyendo el campo protocolo de la cabecera IP

Protocolo detectado en IP: 1

Paso 9: el protocolo detectado es ICMP.
Paso 10: Calculando donde comienza la cabecera ICMP
Paso 11: Leyendo los campos de la cabecera ICMP


ICMP capturado: Tipo=3 Código=13 ID=0 Secuencia=0

Paso 12: guardando el paquete en el archivo icmp_packet.bin
Paso 13: paquete guardado en icmp_packet.bin


Paquete guardado en icmp_packet.bin

Paso 15: Cerrando el socket
Paso 16: Programa finalizado correctamente.
julietabarahonaroth@Julietas-MacBook-Air tp1B %
```

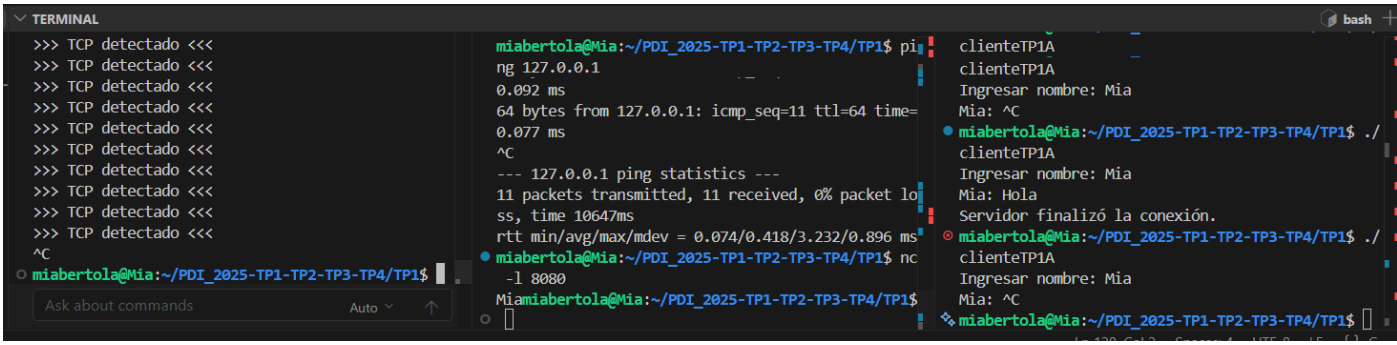
2. código modificado para que capture todo el tráfico:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
```

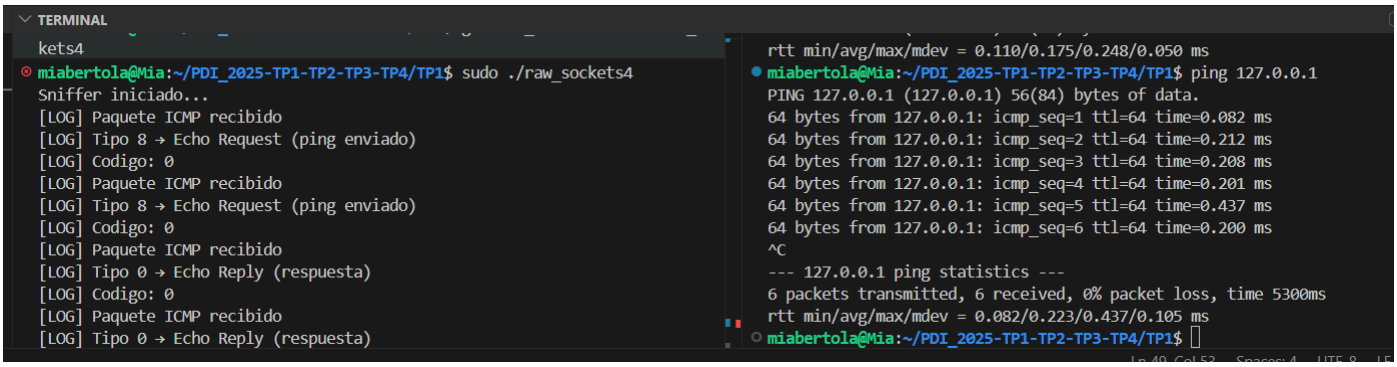
```
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/ip_icmp.h>
// agrego estas tres librerias
#include <netpacket/packet.h>
#include <net/ethernet.h>
#include <arpa/inet.h>

int main() {
   int sockfd;
   char buffer[65536];
   struct sockaddr saddr;
   socklen_t saddr_len = sizeof(saddr);

   // Crear raw socket
   sockfd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL)); // ICMP --> IP todos los
protocolos IP y AD_INET --> AF_PACKET
   if (sockfd < 0) {
       perror("Socket error");
       return 1;
   }

   // acá cambié el mensaje
   printf("Sniffer iniciado (capturando TODO el tráfico)... (Ctrl+C para
detener)\n");

   while (1) {
       ssize_t packet_size = recvfrom(sockfd, buffer, sizeof(buffer), 0, &saddr,
&saddr_len);
       if (packet_size < 0) {
           perror("recvfrom error");
           continue;
       }

       struct iphdr *ip = (struct iphdr*) buffer;
       printf("Paquete recibido. Protocolo: %d\n", ip->protocol); // para ir viendo
los paquetes que llegan
       if (ip->protocol == IPPROTO_ICMP) {
           struct icmphdr *icmp = (struct icmphdr*)(buffer + ip->ihl * 4);
           printf("ICMP capturado: Tipo=%d Código=%d ID=%d Secuencia=%d\n",
               icmp->type,
               icmp->code,
               ntohs(icmp->un.echo.id),
               ntohs(icmp->un.echo.sequence)
           );
```

```
            // Guardar en archivo para analizar con Python
            FILE *f = fopen("icmp_packet.bin", "wb");
            if (f) {
                fwrite(buffer, 1, packet_size, f);
                fclose(f);
                printf("Paquete guardado en icmp_packet.bin\n");
                // break;
            }
        }
    }

    close(sockfd);
    return 0;
}
```

3. Se observó tráfico TCP constante del sistema. Al ejecutar el cliente, se detectó un aumento significativo de paquetes TCP, correspondientes al establecimiento de la conexión y envío de datos. Ajusté el código para observar sólo el tráfico del Cliente.



4. Se generó tráfico ICMP mediante el comando ping a la dirección 127.0.0.1. El sniffer capturó correctamente paquetes de tipo 8 (Echo Request) y tipo 0 (Echo Reply), verificando el intercambio de mensajes entre origen y destino.



5. El sniffer interpreta los paquetes de red mediante estructuras que representan las distintas capas del modelo de comunicación. Primero se analiza la cabecera Ethernet, donde el campo h_proto indica el tipo de protocolo encapsulado, filtrando aquellos correspondientes a IPv4. Luego se accede a la cabecera IP, donde el campo protocol resulta fundamental, ya que indica el protocolo de la capa superior (por ejemplo, 1 para ICMP, 6 para TCP y 17 para UDP). Finalmente, en el caso de ICMP, se analiza su cabecera específica, donde los campos type y code permiten identificar el tipo de mensaje, como Echo Request (tipo 8) y Echo Reply (tipo 0). Esto permite al sniffer interpretar correctamente el tráfico capturado.
6. Se capturaron paquetes donde la comunicación es entre la misma computadora (127.0.0.1). El sniffer detectó que se trata de tráfico TCP, lo que corresponde a la conexión entre el cliente y el servidor y el intercambio de datos entre ellos.

```
[IP] 127.0.0.1 → 127.0.0.1
[TCP] detectado

[IP] 127.0.0.1 → 127.0.0.1
[TCP] detectado

[IP] 127.0.0.1 → 127.0.0.1
[TCP] detectado

[IP] 127.0.0.1 → 127.0.0.1
[TCP] detectado

[IP] 127.0.0.1 → 127.0.0.1
```